

The Lua Programming Language

Introduction

The language I chose for this project is called Lua. Lua is an imperative scripting language that supports both the functional and object oriented paradigms. Lua was designed to be extensible and embeddable; it has a robust C API that allows programmers to extend the language with new types and objects [1, p. xiii]. Furthermore, it is very small: the Linux binary, including the standard library, is only 220 KB [1, p. xiv]. This combination of features have lead to Lua becoming a very popular language to embed in other applications written in C or C++. For instance, I first learned of Lua as the scripting language in which World of Warcraft addons are written, which players can leverage to create custom user interfaces in-game plugins. It is similarly used as an embedded scripting language for configuring popular Linux utilities like NeoVim and nmap.

Lua was designed from the beginning to be embedded in other applications. As a result, the focus of developing the language has been on making it small, fast, and portable. It offers very few basic constructs, but a robust framework for extending the language for domain specific applications. This is to say that Lua is very orthogonal: there are relatively few operators and keywords, as the language itself is tiny. According to the architect of the language, Roberto Ierusalimschy, Lua “does not try to do what C is already good for, such as sheer performance and interface with third-party software. Lua relies on C for these tasks” [1, p. xiii].

The standard Lua runtime is implemented as an interpreter; however, a JIT compiler for Lua is also available.

Grammar and Semantics

Lua reserves the following keywords; they may not be used as names or identifiers.

and	break	do	else	elseif	end
false	for	function	goto	if	in
local	nil	not	or	repeat	return
then	true	until	while		

[1, p. 6]

Valid identifiers in Lua are any string of Latin letters, digits, and underscores. Identifiers may not begin with digits. Lua is case-sensitive, so while “and” is not a valid identifier, “And” is [1, p. 6]

Furthermore, Lua reserves the following tokens as operators:

+	-	*	/	%	^	#
&	~		<<	>>	//	
==	~=	<=	>=	<	>	=
(	)	{	}	[	]	::
;	:	,	.	..	...	

[2, Ch. 3.1]

This provides basic Boolean comparisons, arithmetic operations, exponentiation (“^”), integer & floating point division (“//” and “/”, respectively), as well as bitwise operations (interestingly using “~” for both binary xor and unary not). Semantically, if both operands of an arithmetic operation are integers, then the result will be integers; otherwise, they will be converted to floats and the result will be a float. Two exceptions to this rule are exponentiation and float division, which will always convert their operands to floats and return a float [2, Ch. 3.4.1]. Bitwise operations always convert operands to integers. Bit shift operations only perform logical shifts, filling vacant bits with 0’s [2, Ch. 3.4.2].

Logical operations are implemented using the keywords “and”, “or”, and “not”. Both “and” and “or” operations use short-circuit evaluation.

Below describes the precedence rules for operators in Lua [2, Ch. 3.4.8]:

```

or
and
<      >      <=     >=     ~=     ==
|
~
&
<<     >>
..
+       -
*       /       //      %
unary operators (not  #      -      ~)
^

```

Note that all binary operators are left-associative except for concatenation (“..”) and exponentiation (“^”).

In Lua, variables are dynamically typed. This means that we can simply declare a variable without specifying any type. By default, variables are globally-scoped; they may be declared local which makes them lexically-scoped [2, Ch. 3.2]. Lua permits multiple-assignment, ensuring that if a variable is both read and assigned in an assignment statement, the reads will occur before any assignment [2, Ch. 3.3.3].

Lua provides the following control structures [2, Ch. 3.2]:

```

stat ::= while exp do block end
stat ::= repeat block until exp
stat ::= if exp then block {elseif exp then block} [else block] end
stat ::= for Name '=' exp ',', exp [, exp] do block end
stat ::= for namelist in explist do block end

```

These rules provide basic flow-control, iteration, and conditional looping for programs. Note that Lua provides two for loops: the first one is called the “numerical for-loop”, which iterates over an numeric value as in C. The second is the “generic for-loop”, which allows iteration over an iterator object, which works similarly to for loops in Python. Also note that if

statements must explicitly declare the end of their scope with the “end” keyword. This prevents the dangling-else ambiguity present in C++ and Java.

Types

Lua’s primitive types include Boolean values, nil, numbers, and strings. Nil represents the absence of any value; global variables are nil by default [1, p. 8]. Booleans include values “true” and “false”, but any value can be interpreted as a Boolean. Any value which is not false or nil will be interpreted as true by the Lua interpreter; this includes 0 and the empty string [1, p. 8].

Lua provides a general number type, which consists of 64-bit integers and floating-point numbers. Lua does not distinguish between decimals and integers, except as necessary. For instance, in Lua, the expression $3.0 == 3$ will evaluate as true [1, p. 15]. Lua also supports hexadecimal literals for both integers and floating-point types. For instance, the number 42.75 may be expressed as `0xa.bp2` in Lua, where *p* marks the end of the mantissa and the beginning of the binary exponent [1, pp. 15-16].

In Lua, strings are immutable byte sequences. They are agnostic as to the data encoded, and they can be used to encode binary data, UTF-8, and UTF-16 strings [1, p. 24]. Since strings are treated as objects by the Lua interpreter, they are “subject to automatic memory management” [1, p. 24]. Literal strings may be declared with double quotes, single quotes, or very long strings may be declared with enclosing double-brackets. Lua supports concatenation with the `..` operator. Furthermore, the Lua interpreter will try to coerce strings to numeric values anytime a numeric operation receives a string, and string operations will try to coerce numbers to strings [1, p. 26]. Lua includes a robust string library which includes methods for substrings, format strings, character searching, substitution, and Unicode [1, pp. 27-31].

Tables are the only data structure provided by Lua. Tables can be used to implement lists, sets, dictionaries, records, objects, packages, classes, and any other data structure. Tables are associative arrays. Keys may be string literals, variables, numbers, and any other value except for nil [p. 33]. Tables are dynamically allocated and accessed via references [1, p. 33]. A table constructor is a pair of braces followed by some combination of values and key-value pairs, all separated by commas. Key-value pairs are denoted either with a valid Lua identifier followed by an equal sign, or any arbitrary expression or literal enclosed in brackets. These two syntaxes allow any value (except for nil) to be used as a key. Furthermore, keys may be declared and accessed using a dot notation similar to C structs.

When tables are constructed with values, they are treated as lists. Values will be indexed starting at 1 (not 0 as in other languages). If there are no holes in the data, then the length of the list can be reliably found using the length operator, “#” [1, p. 37].

Tables can be iterated over using either the “pairs” or “ipairs” function. Pairs makes no guarantee of iteration order [1, p. 38]. Ipairs will iterate over the numeric indices of list in increasing order until it finds a nil value. This allows lists to be iterated over like an array in other languages.

Functions

According to the main architect of the language, Roberto Ierumalischy, “Functions in Lua are first-class values”; they may be assigned to variables, store functions as fields in tables, passed as parameters, and returned from functions [1, p. 68]. They use (static) lexical scoping, which allows them to form closures [1, p. 68]. This combined with the fact that all functions are declared anonymously (lambdas) [1, p.68] mean that Lua fully supports functional programming paradigms.

Functions in Lua are declared using the “function” keyword followed by a set of parentheses. The body of a function is terminated with the keyword “end”

Lua functions use positional parameters, and a function call may accept a different number of actual parameters than declared formal parameters without throwing errors. Extra parameters are discarded while missing values are assigned nil [1, p. 42]. Furthermore, Lua supports functions with a variable number of arguments using a “...” syntax [1, p. 45].

Interestingly, functions in Lua may return multiple values, which may be destructured using multiple-assignment syntax [1, pp. 43-44]. This syntax unpacks all the return values into variables, discarding any unused values or assigning nil when there are more variables than values [1, p. 44].

Object Oriented Programming

Lua’s mechanisms for object oriented programming is quite narrow. Primarily, OOP is implemented using metatables and metamethods. A metatable is a regular Lua table that we can set as the prototype for an object [1, p. 152]. When that object is used for various operations, or if we try to access a field of the object which has not been defined, then the behavior of the object may be defined in the metatable [1, p. 152]. An objects metatable may be set using the setmetatable function, or retrieved with the getmetatable function. Metamethods are functions defining behaviors of specific operands [1, p. 152]; they are loosely analogous to magic methods in Python like `__len__`. Some commonly used metamethods include `__add`, `__sub`, `__mul`, `__eq`, `__len`, which define how the underlying object behaves in `+`, `-`, `*`, `==`, and `#` operators respectively.

Perhaps the most important metamethod is the `__index` metamethod. This metamethod defines how an object behaves when a key not defined on the object is referenced [1, pp. 156-157]. Using this feature we can create prototypes (not classes, as Lua has no concept of classes) for objects which store member variables and methods for the object [1, p. 163].

So, using these two features we can define an object A with some member variable foo and some method bar. We can use A as a “class” like so: we create a constructor method on A which sets the `__index` metavalue of A to A itself, then we instantiate a new object o and set the metatable of o to A. Finally, we return o, our newly constructed object which is of class A. What this does is allow o to inherit all of the key-value pairs of A using the `__index` metamethod. If we

tried to access some property `foo` or some method `bar` that `o` has not defined, `foo` and `bar` will be looked up in the metatable of `o`, which is `A`. Because we set the `__index` method of `A` to be itself, all of the members of `A` are available to `o`, and so the Lua interpreter will find definitions of both `foo` and `bar` for `o`.

These metatables can be chained together to create a prototype chain, which effectively enables inheritance. The details of this are quite tedious because Lua does not provide first-class support for this feature, with several implementations defined in [1, Ch. 21, pp. 165-168]. But, it is sufficient to know that Lua can support both single and multiple inheritance through metatables.

One of the other pillars of OOP is information hiding. While Lua does not offer a direct way to mark fields as private, encapsulation can be achieved through closures [1, p. 168]. Particularly, we can hide object state by using closures. If we define all of the state that an object needs as local variables in the constructor function, and we define all of the methods of the object as local variables in the constructor, then we can return an object containing all of the methods, which have closure around the state defined in the constructor [1, pp. 168-169]. In this way, we achieve encapsulation.

Finally, the last pillar of OOP is polymorphism. Lua is a dynamically typed language, so it begins with good support for polymorphism. Through metatables, metavalues, and metamethods in conjunction with Lua's ability to handle variadic functions, we can easily create polymorphic objects and functions.

Conclusion

Lua is a fascinating language which can serve as a powerful template for domain specific languages due to its extensibility. The syntax is clean and clear, and many of the constructs are similar to other dynamic languages like Python. It is relatively small and orthogonal which makes it easy to learn, and even easier to embed in other applications. The one feature it lacks substantial support for is object oriented programming, and though there are ways of using Lua as an object oriented programming language, I find the methods of implementation to be difficult to read and confusing without a very deep understanding of the language. Still, I'd like to practice this language because of the many programs I use on a daily basis which provide APIs accessible via Lua.

References

- [1] R. Ierusalimschy, *Programming in Lua*, Fourth. Rio De Janeiro: Lua.org, 2016.R.
- [2] Ierusalimschy, L. Henrique de Figueiredo, and W. Celes, “Lua 5.4 Reference Manual,” *www.lua.org*, 2023. <https://www.lua.org/manual/5.4/manual.html> (accessed Dec. 12, 2023).